

UNIVERSITÉ D'ANGERS

MASTER MATHÉMATIQUES ET APPLICATIONS

M2 DATA SCIENCE

ANNÉE ACADÉMIQUE 2025–2026

PROJET ANNUEL

Système de recommandation

Étudiants :

Eudes KODIA

Chenny Kessia ISHIMWE

Tuteur enseignant :

Fabien PANLOUP

10 mars 2026

Table des matières

1	Filtrage collaboratif basé sur les méthodes de voisinages	4
1.1	Introduction	4
1.2	Propriétés clés des matrices de notations	5
1.2.1	Effet de la longue traîne	6
1.3	Méthode de voisinage basés sur les utilisateurs	6
1.3.1	Mesure de similarité	7
1.3.2	Variantes de la fonction de prédiction	11
1.4	Méthode de voisinage basés sur les Items	14
1.4.1	Implémentation efficace et Complexité de Calcul	17
1.5	Comparaison entre les méthodes basées sur les utilisateurs et la méthodes basées sur les items	18
1.5.1	Forces et faiblesses des méthodes basées sur les voisinages	19
1.6	Vue unifiée des méthodes basées sur les utilisateurs et les méthodes basées sur les items	20
2	Filtrage collaboratif basé sur les modèles	23
2.1	Introduction	23
2.2	Factorisation de matrices non négatives	23
2.2.1	Formalsation et motivation	23
2.2.2	Problème d’optimisation	24
2.2.3	Structure du problème	25
2.3	Complétion de matrices	28
3	Système de recommandation basé sur l’apprentissage profond	30
3.1	Bases de l’apprentissage profond (ANN)	30
3.2	Filtrage collaboratif neuronal (NCF)	31
3.2.1	Formulation mathématique générale	32
3.2.2	Embeddings utilisateurs et items	33
3.2.3	Architecture du perceptron multicouche (MLP)	33
3.2.4	Fonction de perte et régularisation	34
3.2.5	Interprétation théorique	35

4	Application des méthodes du filtrage collaboratif	36
4.1	Comparaison expérimentale des méthodes de recommandation	36
4.2	Résultats obtenus	36
4.3	Analyse des résultats	37
	Conclusion générale	39

Introduction

Les systèmes de recommandation occupent aujourd’hui une place centrale dans de nombreux services numériques : plateformes de streaming (Netflix, YouTube), sites de commerce (Amazon), réseaux sociaux (Facebook, TikTok), moteurs de recherche, ou encore portails d’actualités. Leur objectif est de filtrer efficacement une grande quantité d’informations afin de proposer à chaque utilisateur des contenus susceptibles de correspondre à ses préférences. Parmi les différentes familles de techniques existantes, le filtrage collaboratif demeure l’une des approches les plus utilisées en raison de sa simplicité conceptuelle et de sa capacité à exploiter directement les comportements collectifs des utilisateurs. L’idée centrale est que des utilisateurs ayant aimé des contenus similaires dans le passé auront tendance à partager des préférences futures. Ce projet se concentre exclusivement sur les modèles de filtrage collaboratif. Après avoir présenté les concepts fondamentaux, nous introduirons les principales mesures de similarité, les méthodes basées sur la mémoire, ainsi que leurs variantes. Le tableau ci-dessous énumère certains exemples de plateformes utilisant des systèmes de recommandation basés sur le filtrage collaboratif.

Tableau 1.1 : Exemples de produits recommandés par divers systèmes de recommandation réels

Système	Produit / Objectif
Amazon.com	Livres et autres produits
Netflix	DVD, vidéos en streaming
Jester	Blagues
GroupLens	Actualités
MovieLens	Films
last.fm	Musique
Google News	Actualités
Google Search	Publicités
Facebook	Amis, publicités
Pandora	Musique
YouTube	Vidéos en ligne
Tripadvisor	Produits liés au voyage
IMDb	Films

Chapitre 1

Filtrage collaboratif basé sur les méthodes de voisinages

1.1 Introduction

Les algorithmes de filtrage collaboratif basés sur les méthodes de voisinages, aussi appelés *algorithmes basés sur la mémoire*, comptent parmi les premiers modèles développés pour le filtrage collaboratif. Ils reposent sur l'idée que des utilisateurs similaires affichent des comportements de notation semblables, et que des objets similaires reçoivent des évaluations proches.

Il existe deux grandes catégories de méthodes basées sur les voisinages :

- **Filtrage collaboratif basé sur les utilisateurs** : les évaluations fournies par les utilisateurs similaires à un utilisateur cible A servent à prédire les notes de A . Les prédictions sont calculées comme une moyenne pondérée des notes de ce groupe de pairs.
- **Filtrage collaboratif basé sur les items** : pour recommander un item cible B , on détermine d'abord un ensemble S d'items similaires à B . La prédiction de la note qu'un utilisateur A donnerait à B s'obtient par la moyenne pondérée des notes que A a attribuées aux items de l'ensemble S .

La principale différence entre ces deux approches est que :

- Dans le **modèle utilisateur-utilisateur**, les prédictions sont basées sur les notes d'autres utilisateurs similaires.
- Dans le **modèle item-item**, les prédictions proviennent des propres notes de l'utilisateur sur des items voisins.

Les deux approches sont donc complémentaires, bien qu'elles produisent des résultats différents.

Comme dans tout système de recommandation, on considère une matrice d'évaluations $R = [r_{uj}]$ de dimensions $m \times n$ contenant m utilisateurs et n items. La plupart des entrées sont manquantes.

Deux formulations sont possibles :

1. **Prédiction de notes manquantes** : on estime directement la valeur manquante r_{uj} .
2. **Détermination des top- k items (ou utilisateurs)** : dans la pratique, les systèmes affichent une liste d'items les plus pertinents plutôt que des valeurs numériques de notes. Pour obtenir ces top- k items, on peut prédire les notes puis les trier.

Les méthodes basées sur les voisinages pré-calculent une partie des informations (similarités, voisinages) en **phase hors-ligne**, afin d'accélérer la recommandation en **phase en ligne**.

Dans ce chapitre, on présentera :

- les propriétés importantes des matrices de notation ;
- les algorithmes utilisateur-utilisateur et item-item ;
- la réduction de dimension pour améliorer les performances ;
- une vue unifiée via un modèle de régression

1.2 Propriétés clés des matrices de notations

On considère une matrice de notations R de dimension $m \times n$ contenant m utilisateurs et n items. La majorité des entrées de la matrice sont manquantes. Les entrées observées constituent les *données d'entraînement*, tandis que les entrées manquantes constituent les *données de test*. Cette configuration est analogue à la classification ou la régression, dans lesquelles les valeurs inconnues correspondent aux variables dépendantes.

Les notations peuvent être définies de plusieurs façons, selon l'application :

- **Notations continues** : les évaluations sont exprimées sur une échelle réelle continue, par exemple entre -10 et 10 dans le système Jester. Ce type d'évaluation est rare, car il impose une charge cognitive importante à l'utilisateur.
- **Notations par intervalles** : les évaluations proviennent d'une échelle discrète, typiquement de 1 à 5 ou de 1 à 7 . Les valeurs expriment explicitement des distances égales entre les niveaux.
- **Notations ordinales** : les évaluations correspondent à des catégories ordonnées (*Fortement en désaccord*, *Désaccord*, etc.). En pratique, ces catégories sont souvent converties en valeurs numériques supposées équidistantes, rapprochant ainsi cette approche du modèle par intervalles.

- **Notations binaires** : l'utilisateur fournit uniquement un vote positif ou négatif, par exemple aimer ou ne pas aimer un morceau musical. Les valeurs neutres ne sont pas exprimées (l'absence d'évaluation pouvant être interprétée comme une neutralité).
- **Notations unaires** : l'utilisateur ne peut exprimer qu'un intérêt positif explicite, comme un «J'aime ». L'absence d'action n'implique pas nécessairement un désintérêt. Les notations unaires correspondent fréquemment à du *feedback implicite*, issu du comportement utilisateur (clic, achat, écoute, etc.).

1.2.1 Effet de la longue traîne

Dans de nombreux scénarios, la distribution des notations suit une **longue traîne**. Quelques items reçoivent énormément de notations (items populaires), tandis que la majorité sont très peu notés.

Cette distribution entraîne plusieurs conséquences :

1. Les items populaires sont souvent peu rentables pour le marchand, alors que les items de la longue traîne possèdent une marge plus élevée.
2. Les algorithmes préfèrent souvent les items populaires, ce qui nuit à la diversité des recommandations.
3. Les items fréquemment notés dominent les calculs de voisinage, ce qui produit des recommandations moins pertinentes pour les items rares.

Il est donc essentiel d'ajuster les méthodes de recommandation afin de tenir compte de la sparsité et de la longue traîne, pour améliorer la pertinence des prédictions.

1.3 Méthode de voisinage basés sur les utilisateurs

Dans ce type d'approche, on détermine un voisinage d'utilisateurs similaires à l'utilisateur cible i , pour lequel on souhaite prédire des notes. Pour cela, on calcule la similarité entre l'utilisateur cible et tous les autres utilisateurs. Une fonction de similarité doit donc être définie entre les vecteurs de notation des utilisateurs.

Cette opération est délicate pour deux raisons principales :

- les utilisateurs ont des **échelles de notation différentes** (certains notent généreusement, d'autres sévèrement) ;
- les utilisateurs n'ont pas évalué les mêmes items.

Considérons une matrice $R = [r_{uj}]$ de dimension $m \times n$. On note I_u l'ensemble des indices des items évalués par l'utilisateur u . Ainsi, l'ensemble des items évalués par *deux* utilisateurs u et v est :

$$I_{uv} = I_u \cap I_v.$$

Cet ensemble peut être vide, car les matrices de notation sont très creuses.

1.3.1 Mesure de similarité

Coefficient de corrélation de Pearson

Pour identifier des utilisateurs similaires, il est nécessaire de définir une mesure de similarité entre leurs vecteurs de notation. Parmi les différentes mesures possibles, la corrélation de Pearson est l'une des plus couramment utilisées dans les approches de filtrage collaboratif basées sur les utilisateurs ; elle permet de quantifier la proximité entre deux utilisateurs u et v en étant calculée exclusivement sur l'ensemble des items qu'ils ont évalués en commun.

On définit d'abord la moyenne des évaluations de l'utilisateur u :

$$\mu_u = \frac{1}{|I_u|} \sum_{j \in I_u} r_{uj}.$$

La corrélation de Pearson entre les utilisateurs u et v est alors :

$$\text{Pearson}(u, v) = \frac{\sum_{j \in I_{uv}} (r_{uj} - \mu_u)(r_{vj} - \mu_v)}{\sqrt{\sum_{j \in I_{uv}} (r_{uj} - \mu_u)^2} \sqrt{\sum_{j \in I_{uv}} (r_{vj} - \mu_v)^2}}.$$

Dans la pratique, on calcule souvent μ_u une seule fois pour chaque utilisateur, plutôt que pour chaque paire (u, v) .

Prédiction par moyenne pondérée

Pour prédire la note de l'utilisateur u pour l'item j , on sélectionne les k utilisateurs les plus similaires ayant évalué j . On note cet ensemble :

$$P_u(j).$$

Les notes sont d'abord **centrées** en retirant la moyenne de chaque utilisateur :

$$s_{uj} = r_{uj} - \mu_u.$$

La prédiction finale est :

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) s_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}.$$

Remarques :

- Les utilisateurs faiblement ou négativement corrélés sont parfois éliminés du voisinage pour éviter de dégrader la prédiction.
- La normalisation par centrage améliore notablement la cohérence des prédictions.

Cosinus brut sur les notes non centrées

Au lieu d'utiliser la corrélation de Pearson, on peut employer la similarité cosinus sur les notes brutes :

$$\text{Cosine}(u, v) = \frac{\sum_{j \in I_{uv}} r_{uj} r_{vj}}{\sqrt{\sum_{j \in I_{uv}} r_{uj}^2} \sqrt{\sum_{j \in I_{uv}} r_{vj}^2}}.$$

Certaines implémentations utilisent l'ensemble complet des notes spécifiées par chaque utilisateur dans le dénominateur, plutôt que les seuls items en commun :

$$\text{CosineAlt}(u, v) = \frac{\sum_{j \in I_{uv}} r_{uj} r_{vj}}{\sqrt{\sum_{j \in I_u} r_{uj}^2} \sqrt{\sum_{j \in I_v} r_{vj}^2}}.$$

La corrélation de Pearson tend toutefois à donner de meilleurs résultats.

Une fois la mesure de similarité définie, il devient possible de construire un voisinage d'utilisateurs pertinents afin d'estimer les notes manquantes.

Exemple d'Algorithme basé sur les utilisateurs

Considérons l'exemple du Tableau 1.1. Dans ce cas, les notes de cinq utilisateurs (1 à 5) sont indiquées pour six items (1 à 6). Chaque note est tirée de l'intervalle $\{1, \dots, 7\}$. Considérons le cas où l'utilisateur cible est l'utilisateur 3, et où nous souhaitons produire des prédictions d'items sur la base des notes du Tableau 1.1.

Nous devons calculer les prédictions pour l'utilisateur 3 concernant les items 1 et 6 afin de déterminer quel item doit être recommandé en priorité.

La première étape consiste à calculer la similarité entre l'utilisateur 3 et les autres utilisateurs. Nous avons montré deux façons possibles de calculer la similarité dans les

TABLE 1.1 – Calcul de similarité utilisateur-utilisateur entre l'utilisateur 3 et les autres utilisateurs

User-Id	1	2	3	4	5	6	Mean	Cosine(i,3)	Pearson(i,3)
1	7	6	7	4	5	4	5.5	0.956	0.894
2	6	7	?	4	3	4	4.8	0.981	0.939
3	?	3	3	1	1	?	2	1.0	1.0
4	1	2	2	3	3	4	2.5	0.789	-1.0
5	1	?	1	2	3	3	2	0.645	-0.817

deux dernières colonnes du tableau :

- l'avant-dernière colonne utilise la similarité cosinus brute appliquée aux notes,
- la dernière colonne utilise la corrélation de Pearson.

Par exemple, les valeurs de $\text{Cosine}(1, 3)$ et $\text{Pearson}(1, 3)$ sont calculées comme suit (en utilisant uniquement les items notés en commun) :

$$\text{Cosine}(3, 1) = \frac{3 \cdot 6 + 3 \cdot 7 + 1 \cdot 4 + 1 \cdot 5}{\sqrt{3^2 + 3^2 + 1^2 + 1^2} \sqrt{6^2 + 7^2 + 4^2 + 5^2}} = \frac{18 + 21 + 4 + 5}{\sqrt{20} \sqrt{126}} = \frac{48}{\sqrt{2520}} \approx 0.956.$$

$$\text{Pearson}(3, 1) = \frac{\sum_j (r_{3j} - \mu_3)(r_{1j} - \mu_1)}{\sqrt{\sum_j (r_{3j} - \mu_3)^2} \sqrt{\sum_j (r_{1j} - \mu_1)^2}} \approx 0.894.$$

Les similarités cosinus et de Pearson entre l'utilisateur 3 et les autres utilisateurs sont indiquées dans les deux dernières colonnes du Tableau 1.1. Notez que le coefficient de Pearson est nettement plus discriminant, et que son signe donne directement une information de similarité ou dissimilarité.

Les deux utilisateurs les plus proches de l'utilisateur 3 sont les utilisateurs 1 et 2, selon les deux mesures. À l'aide de la moyenne pondérée par Pearson des notes brutes des utilisateurs 1 et 2, on obtient les prédictions suivantes pour l'utilisateur 3 concernant ses items non notés 1 et 6 :

$$\hat{r}_{3,1} = \frac{0.894 \cdot 7 + 0.939 \cdot 6}{|0.894| + |0.939|} = \frac{6.261 - (\text{approx}) + 5.632098}{0.894 + 0.939} = \frac{11.893098}{1.83311019} \approx 6.49.$$

$$\hat{r}_{3,6} = \frac{0.894 \cdot 4 + 0.939 \cdot 4}{|0.894| + |0.939|} = \frac{(0.89442719 + 0.938683) \cdot 4}{1.83311019} = \frac{1.83311019 \cdot 4}{1.83311019} = 4$$

Ainsi, l'item 1 doit être priorisé par rapport à l'item 6 dans la recommandation à l'utilisateur 3.

De plus, la prédiction suggère que l'utilisateur 3 serait susceptible d'aimer les items 1

et 6 davantage que tous les items qu'elle a déjà notés. Cependant, cela provient du biais causé par le fait que le groupe de pairs $\{1, 2\}$ est nettement plus optimiste (notes plus élevées) que l'utilisateur cible 3.

Examinons maintenant l'impact du *centrage par la moyenne*. Les notes centrées sont indiquées dans le Tableau 1.2.

Avec les équations centrées, les prédictions deviennent :

$$\hat{r}_{3,1} = 2 + \frac{1.5 \cdot 0.89 + 1.2 \cdot 0.939}{0.89 + 0.939} = 2 + \frac{2.4617999999999998}{1.829} \approx 3.35.$$

$$\hat{r}_{3,6} = 2 + \frac{-1.5 \cdot 0.89 + 0.8 \cdot 0.939}{0.89 + 0.939} = 2 + \frac{-2.0922}{1.829} \approx 0.86.$$

Puis, on revient à l'échelle brute en ajoutant μ_3 .

Dans ce cas, on obtient à nouveau que l'item 1 doit être recommandé avant l'item 6. Cependant, il existe une différence cruciale par rapport au cas précédent : la prédiction de l'item 6 est seulement de 0.86, ce qui est inférieur à toutes les autres notes de l'utilisateur 3. C'est très différent du cas précédent, où l'item 6 avait été prédit au-dessus de toutes les notes existantes de l'utilisateur 3.

En inspectant visuellement les Tableaux 1.1 et 1.2, il est clair que l'item 6 devrait être noté très bas par l'utilisateur 3, puisque ses deux plus proches pairs (1 et 2) lui donnent également une note plus faible que les autres items.

Ainsi, le centrage par la moyenne permet une prédiction beaucoup plus cohérente avec les notes observées. Il peut également modifier le classement relatif des items.

La seule faiblesse est que la prédiction centrée peut tomber en dehors de l'ensemble des valeurs acceptables (ex. : 0.85). Dans ce cas, la valeur peut être utilisée pour le classement, puis tronquée à la valeur autorisée la plus proche.

TABLE 1.2 – Matrice de notes du Tableau 2.1 avec centrage par la moyenne pour le calcul du cosinus ajusté entre items. Les similarités ajustées des items 1 et 6 avec les autres items sont montrées dans les dernières lignes.

User-Id	1	2	3	4	5	6
1	1.5	0.5	1.5	2	1.2	2.2
2	-1.5	-0.5	-1.5	?	3	?
3	?	1	-0.8	-1.8	-0.8	1
4	-1.5	-0.5	-1	-1	?	-0.5
5	-1	?	0.5	0.5	1.5	-1
Cosine(1,j)	1	0.735	0.912	-0.848	-0.813	-0.990
Cosine(6,j)	-0.990	-0.622	-0.912	0.829	0.730	1

Pondération par significativité

La fiabilité de la similarité dépend fortement du nombre d'items évalués en commun $|I_{uv}|$. Si ce nombre est trop faible, la similarité doit être diminuée.

On introduit donc un facteur de pondération :

$$w_{uv} = \begin{cases} \frac{|I_{uv}|}{\beta}, & \text{si } |I_{uv}| < \beta, \\ 1, & \text{sinon,} \end{cases}$$

où β est un seuil (souvent 50).

La similarité ajustée devient :

$$\text{DiscountedSim}(u, v) = w_{uv} \cdot \text{Sim}(u, v).$$

Cette version pondérée est ensuite utilisée à la fois pour :

- sélectionner le voisinage $P_u(j)$,
- effectuer la prédiction via la moyenne pondérée.

1.3.2 Variantes de la fonction de prédiction

Diverses variantes de la formule de prédiction basée sur les utilisateurs peuvent être utilisées. Deux types de modifications sont particulièrement courants : la normalisation des notes et le réajustement des poids.

1. Normalisation par Z-score

Au lieu du centrage simple,

$$s_{uj} = r_{uj} - \mu_u,$$

on peut appliquer une normalisation par écart-type, appelée *Z-score* :

$$\sigma_u = \sqrt{\frac{1}{|I_u|} \sum_{j \in I_u} (r_{uj} - \mu_u)^2}.$$

Le score normalisé devient alors :

$$z_{uj} = \frac{r_{uj} - \mu_u}{\sigma_u}.$$

La prédiction centrée-normalisée est :

$$\hat{z}_{uj} = \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) z_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}.$$

Enfin, on revient à l'échelle originale en appliquant l'inverse :

$$\hat{r}_{uj} = \mu_u + \sigma_u \hat{z}_{uj}.$$

Cette méthode améliore parfois la qualité des recommandations, mais elle peut produire des valeurs prédites en dehors de l'intervalle possible des notes. On utilise alors ces valeurs pour le classement, quitte à les tronquer ensuite.

2. Amplification des Similarités

Les formules de prédiction utilisent des coefficients de similarité comme poids :

$$\text{poids}(u, v) = \text{Sim}(u, v).$$

Une pratique courante consiste à amplifier l'effet des similarités élevées via une exponentiation :

$$\text{poids}(u, v) = (\text{Sim}(u, v))^\alpha, \quad \alpha > 1.$$

Un α plus grand renforce l'influence des utilisateurs très similaires et réduit celle des utilisateurs moyennement similaires.

3. Approche de type classification

Les prédictions précédentes traitent la note comme une variable continue (régression). Une autre approche consiste à considérer les notes comme des catégories (1, 2, 3, 4, 5 ou « d'accord », « neutre », « pas d'accord »).

Une fois le voisinage $P_u(j)$ identifié, on compte le nombre de votes pour chaque valeur de note et l'on choisit la plus fréquente :

$$\hat{r}_{uj} = \arg \max_c |\{v \in P_u(j) : r_{vj} = c\}|.$$

Cette méthode :

- fonctionne bien lorsque le nombre de valeurs possibles est faible ;
- est adaptée aux notations ordinales ;
- mais perd beaucoup d'information lorsque les notes sont plus fines.

Variations dans la sélection des Groupes de Pairs

Le groupe de pairs d'un utilisateur cible peut être défini et filtré de multiples manières. L'approche la plus simple consiste à sélectionner les k utilisateurs les plus similaires. Toutefois, cette méthode peut inclure des utilisateurs :

- faiblement corrélés,
- ou même négativement corrélés.

Ces utilisateurs peuvent dégrader la qualité des prédictions, car :

- les corrélations faibles introduisent du bruit,
- les corrélations négatives ne sont généralement pas pertinentes dans le cadre des méthodes fondées sur le voisinage.

Même si la formule de prédiction *permet* théoriquement des similarités faibles ou négatives, celles-ci ne respectent pas l'esprit du modèle de voisinage, qui repose sur l'idée de comportements proches.

Ainsi, il est courant :

- d'imposer un seuil minimal de similarité,
- d'exclure les corrélations négatives,
- ou d'inclure uniquement les utilisateurs ayant un nombre suffisant d'items communs avec l'utilisateur cible.

Ces filtrages améliorent généralement la stabilité et la pertinence des recommandations.

Impact de la Longue Traîne

Comme discuté dans la section 1.2.1, les distributions de notations observées dans la plupart des systèmes réels suivent une **longue traîne**. Certains items sont évalués extrêmement fréquemment, tandis que la grande majorité le sont très rarement.

Cet effet a un impact direct sur les méthodes de voisinage basées sur les utilisateurs.

Problèmes causés par les items très populaires Les items populaires apparaissent souvent dans les ensembles d'items évalués par de nombreux utilisateurs. Cependant :

- ils sont **peu discriminants** : presque tout le monde les a vus ou notés ;
- ils dominent la similarité entre utilisateurs ;
- ils conduisent parfois à des recommandations répétitives et sans variété.

Cette situation est analogue au rôle des mots très fréquents (« the », « a ») dans la recherche documentaire, qui détériorent la qualité du matching.

Un remède : la fréquence inverse des utilisateurs (IUF) Inspiré de l'*inverse document frequency (idf)* en recherche d'information, on peut pondérer les items en fonction de leur rareté.

Si m est le nombre total d'utilisateurs et m_j le nombre d'utilisateurs ayant noté l'item j , on définit :

$$w_j = \log \frac{m}{m_j}.$$

Un item rare (faible m_j) reçoit un poids élevé, tandis qu'un item populaire (représenté partout) reçoit un poids faible.

Ce poids peut être intégré :

- dans le calcul de similarité entre utilisateurs,
- dans la prédiction elle-même,
- ou dans les deux.

Exemple avec la corrélation de Pearson pondérée La version pondérée devient :

$$\text{Pearson}_{\text{IUF}}(u, v) = \frac{\sum_{j \in I_{uv}} w_j (r_{uj} - \mu_u)(r_{vj} - \mu_v)}{\sqrt{\sum_{j \in I_{uv}} w_j (r_{uj} - \mu_u)^2} \sqrt{\sum_{j \in I_{uv}} w_j (r_{vj} - \mu_v)^2}}.$$

Effet global Cette correction rend les similarités plus sensibles aux items rares, donc plus informatives, améliorant ainsi :

- la diversité des recommandations,
- la qualité des prédictions,
- la pertinence des voisins sélectionnés.

1.4 Méthode de voisinage basés sur les Items

Dans les modèles basés sur les items, les voisinages sont formés entre items (colonnes de la matrice de notation) plutôt qu'entre utilisateurs. L'objectif est de trouver les items les plus similaires à un item cible, puis d'utiliser les notes déjà attribuées par l'utilisateur à ces items similaires pour prédire la note manquante.

Centrage préalable des notes Avant de calculer la similarité entre deux items, chaque ligne de la matrice (référant à un utilisateur) est centrée autour de sa moyenne :

$$s_{uj} = r_{uj} - \mu_u.$$

Cela corrige les différences de niveau entre utilisateurs.

On note U_i l'ensemble des utilisateurs ayant évalué l'item i .

Similarité cosinus ajustée La mesure de similarité la plus courante pour les modèles item-item est la **similarité cosinus ajustée**, définie par :

$$\text{AdjustedCosine}(i, j) = \frac{\sum_{u \in U_i \cap U_j} s_{ui} s_{uj}}{\sqrt{\sum_{u \in U_i \cap U_j} s_{ui}^2} \sqrt{\sum_{u \in U_i \cap U_j} s_{uj}^2}}.$$

Cette mesure est préférée au coefficient de Pearson dans le cas des items, car le centrage a déjà été effectué par utilisateur, rendant le cosinus suffisant.

Prédiction Pour prédire la note manquante de l'utilisateur u pour l'item cible t :

1. on détermine les k items les plus similaires à t ,
 2. parmi ceux-ci, on sélectionne ceux que l'utilisateur u a évalués ;
- cet ensemble est noté :

$$Q_t(u).$$

La prédiction est une moyenne pondérée :

$$\hat{r}_{ut} = \frac{\sum_{j \in Q_t(u)} \text{AdjustedCosine}(t, j) r_{uj}}{\sum_{j \in Q_t(u)} |\text{AdjustedCosine}(t, j)|}.$$

Interprétation Cette méthode exploite les **préférences passées** de l'utilisateur lui-même. Par exemple, pour un film historique, les films similaires déjà évalués par l'utilisateur permettent de prédire avec précision son appréciation du film cible. Cela conduit souvent à des prédictions :

- plus stables,
- plus cohérentes,
- moins sensibles au bruit entre utilisateurs.

Les variantes des fonctions de prédiction et de similarité présentées dans la section précédente (utilisateur-utilisateur) peuvent être transposées de manière analogue au cas item-item.

Exemple d'algorithme basé sur les items

Afin d'illustrer l'algorithme basé sur les items, nous utiliserons le même exemple que celui du Tableau 2.1, qui a été utilisé pour démontrer l'algorithme basé sur les utilisateurs.

Les notes manquantes de l'utilisateur 3 sont prédites à l'aide de l'algorithme basé sur les items. Comme les notes des items 1 et 6 sont manquantes pour l'utilisateur 3, la similarité des colonnes correspondant aux items 1 et 6 doit être calculée par rapport aux autres colonnes (items).

Tout d'abord, la similarité entre les items est calculée après avoir centré les notes par rapport à leur moyenne. La matrice des notes centrées est illustrée dans le Tableau 2.2. Les similarités cosinus ajustées de chaque item par rapport aux items 1 et 6, respectivement, sont indiquées dans les deux dernières lignes du tableau. Par exemple, la valeur du cosinus ajusté entre les items 1 et 3, notée $\text{AdjustedCosine}(1,3)$, est la suivante :

$$\begin{aligned}\text{AdjustedCosine}(1,3) &= \frac{(6 - 3.75)(3 - 2.5) + (1 - 3.75)(2 - 2.5)}{\sqrt{(6 - 3.75)^2 + (1 - 3.75)^2} \sqrt{(3 - 2.5)^2 + (2 - 2.5)^2}} \\ &= \frac{2.25 \cdot 0.5 + (-2.75)(-0.5)}{\sqrt{2.25^2 + (-2.75)^2} \sqrt{0.5^2 + (-0.5)^2}} \\ &= 0.912.\end{aligned}$$

Les autres similarités item-item sont calculées de manière exactement analogue et sont illustrées dans les deux dernières lignes du Tableau 2.2. Il est évident que les items 2 et 3 sont les plus similaires à l'item 1, tandis que les items 4 et 5 sont les plus similaires à l'item 6. Par conséquent, la moyenne pondérée des notes brutes de l'utilisateur 3 pour les items 2 et 3 est utilisée pour prédire la note de l'item 1, tandis que la moyenne pondérée des notes brutes de l'utilisateur 3 pour les items 4 et 5 est utilisée pour prédire la note de l'item 6 :

$$r_{3,1} = \frac{\text{sim}(1,2) \cdot r_{3,2} + \text{sim}(1,3) \cdot r_{3,3}}{|\text{sim}(1,2)| + |\text{sim}(1,3)|} = \frac{0.735 \cdot 3 + 0.912 \cdot 3}{0.735 + 0.912} = \frac{3.924 + 2.988}{0.981 + 0.996} = \frac{6.912}{1.977} \approx 3$$

Ainsi, la méthode basée sur les items suggère également que l'item 1 est plus susceptible d'être préféré par l'utilisateur 3 que l'item 6. Cependant, dans ce cas, comme les notes sont prédites en utilisant les notes de l'utilisateur 3 lui-même, les notes prédites ont tendance à être beaucoup plus cohérentes avec les autres notes de cet utilisateur. À titre d'exemple, il est à noter que la note prédite pour l'item 6 ne se situe plus en dehors de la plage de notes autorisées, comme c'était le cas avec la méthode basée sur les utilisateurs. La plus grande précision de prédiction de la méthode basée sur les items constitue son principal avantage. Dans certains cas, la méthode basée sur les items peut fournir un ensemble différent de recommandations top-k, même si les listes recommandées seront généralement approximativement similaires.

1.4.1 Implémentation efficace et Complexité de Calcul

Les méthodes fondées sur les voisinages servent à déterminer :

- les meilleurs items à recommander à un utilisateur cible,
- ou les meilleurs utilisateurs à cibler pour un item donné.

Les sections précédentes ont décrit comment prédire une note utilisateur-item particulière, mais pas comment classer efficacement les items ou les utilisateurs. Une approche naïve serait de prédire toutes les notes possibles, puis les trier, ce qui peut être coûteux.

Heureusement, la prédiction réutilise de nombreuses quantités intermédiaires. Ainsi, une séparation entre phase hors-ligne et phase en ligne est utilisée.

Phase hors-ligne

Dans cette phase, on pré-calcule :

- les similarités utilisateur-utilisateur ou item-item,
- les groupes de pairs associés à chaque utilisateur ou item.

Ces données sont ensuite stockées pour utilisation directe lors des requêtes.

Notons :

$$n' = \max_u |I_u|, \quad m' = \max_j |U_j|,$$

où n' est le maximum d'items notés par un utilisateur et m' le maximum d'utilisateurs ayant noté un item.

Coûts hors-ligne

- **Méthodes basées sur les utilisateurs** : déterminer les groupes de pairs nécessite un temps :

$$O(m \cdot n'),$$

donc un coût total :

$$O(m^2 \cdot n').$$

- **Méthodes item-item** : le coût hors-ligne est :

$$O(n^2 \cdot m').$$

Mémoire Pour pouvoir utiliser différentes valeurs de k , on stocke généralement *toutes* les similarités non nulles entre paires :

$$\text{Méthodes utilisateur : } O(m^2), \quad \text{Méthodes item : } O(n^2).$$

Comme le nombre d'utilisateurs est souvent bien supérieur au nombre d'items, les méthodes item-item sont généralement plus économes.

Phase en ligne

Pour une prédiction utilisant un voisinage de taille k :

$$O(k).$$

Pour recommander tous les items à un utilisateur (classement complet) :

$$O(k \cdot n).$$

Pour recommander les meilleurs utilisateurs pour un item (ciblage marketing) :

$$O(k \cdot m).$$

Observation importante La principale complexité réside dans la phase hors-ligne, exécutée ponctuellement. La phase en ligne reste très rapide, ce qui rend ces méthodes efficaces dans des systèmes à grande échelle une fois la préparation effectuée.

1.5 Comparaison entre les méthodes basées sur les utilisateurs et la méthodes basées sur les items

Les méthodes basées sur les items produisent souvent des recommandations plus précises que les méthodes basées sur les utilisateurs. Cela tient au fait que les prédictions reposent sur les **propres évaluations de l'utilisateur** sur des items similaires, plutôt que sur les évaluations d'autres utilisateurs potentiellement différents dans leurs goûts.

Avantage des méthodes item-based Pour un item cible t , les items les plus similaires sont souvent très liés (genre, style, auteur, thème...). Les notes que l'utilisateur a attribuées à ces items proches sont de bons indicateurs de sa préférence pour t .

En revanche, dans les méthodes basées sur les utilisateurs, les prédictions sont dérivées à partir d'autres utilisateurs, dont les goûts se recoupent partiellement mais pas totalement, ce qui peut introduire du bruit.

Les méthodes item-based sont donc souvent :

- plus précises ;
- plus stables face aux variations dans les notes ;
- plus robustes face aux attaques de type *shilling*.

Diminution de la diversité Cependant, cette forte précision peut réduire la diversité. Les items similaires sont souvent trop proches, ce qui conduit à :

- recommander des items trop attendus ;
- manquer de surprise ou de découverte ;
- proposer moins de variété dans la liste finale.

C'est l'une des raisons pour lesquelles les méthodes utilisateur-based peuvent générer davantage de **diversité** et parfois plus de **sérendipité**.

Explicabilité Les méthodes item-based permettent souvent de fournir des explications simples et convaincantes :

*« Parce que vous avez regardé *Secrets of the Wings*... »*

L'item cible est expliqué via les similarités entre items, ce qui est naturel.

Les méthodes basées sur les utilisateurs sont moins explicables, car le voisinage est composé d'« utilisateurs anonymes » dont les identités ne peuvent pas être révélées pour des raisons de confidentialité.

Un exemple d'explication use-based consiste à afficher un histogramme des notes données par les voisins, mais cela ne relie pas directement la recommandation aux préférences personnelles de l'utilisateur.

Stabilité des modèles Les méthodes item-based sont plus stables pour deux raisons :

1. **Le nombre d'items est bien plus faible** que le nombre d'utilisateurs. Deux items ont plus de chances d'avoir été co-notés par un grand nombre d'utilisateurs, ce qui rend les similarités item-item plus fiables.
2. **L'arrivée de nouveaux utilisateurs est fréquente**, mais les items évoluent plus lentement. Le recalcul des voisinages utilisateur-basés doit donc être plus fréquent.

Ainsi, les méthodes item-based sont plus faciles à maintenir dans des systèmes de grande taille.

1.5.1 Forces et faiblesses des méthodes basées sur les voisinages

Les méthodes de voisinage possèdent plusieurs avantages importants, liés à leur simplicité et à leur caractère intuitif.

Forces

- **Simplicité d'implémentation** : les formules sont directes, faciles à coder et à déboguer.

- **Interprétabilité** : il est possible d'expliquer clairement pourquoi un item est recommandé. Les méthodes item-based sont particulièrement explicables.
- **Stabilité des recommandations** : elles restent relativement robustes avec l'ajout de nouveaux utilisateurs ou de nouveaux items.
- **Maintenance incrémentale possible** : on peut mettre à jour progressivement les voisinages ou les similarités.

Faiblesses

- **Phase hors-ligne coûteuse** : calculer les similarités utilisateur-utilisateur nécessite au minimum

$$O(m^2)$$

en temps et en mémoire, ce qui devient prohibitif pour des millions d'utilisateurs.

- **Couverture limitée en raison de la sparsité** : si aucun voisin de John n'a noté *Terminator*, il est impossible de prédire la note que John donnerait à ce film. Heureusement, dans la pratique, on recommande surtout les top- k items, ce qui réduit l'impact de ce problème.
- **Similarités peu fiables lorsque les notes en commun sont rares** :
- la matrice de notation étant creuse, deux utilisateurs peuvent n'avoir qu'un nombre très faible d'items co-notés, rendant les similarités instables.

Ainsi, bien que les méthodes de voisinage soient simples et efficaces en ligne, elles souffrent de limitations liées à la sparsité et à la complexité, computationnelle hors-ligne.

1.6 Vue unifiée des méthodes basées sur les utilisateurs et les méthodes basées sur les items

Les méthodes de voisinage user-based et item-based semblent différentes, mais elles peuvent être formulées dans un cadre mathématique **unique**. Cette vision unifiée repose sur l'idée que les prédictions sont toujours une combinaison linéaire pondérée de notes déjà observées.

Formulation générale On cherche à prédire la valeur manquante r_{uj} pour un utilisateur u et un item j .

On note :

$$\hat{r}_{uj} = a_{uj} + \sum_{(v,i) \in N_{uj}} b_{uj}(v,i) r_{vi},$$

où :

- N_{uj} est l'ensemble des paires (utilisateur, item) utilisées pour prédire la note manquante,
- a_{uj} est un terme d'ajustement ou de normalisation,
- $b_{uj}(v, i)$ est le poids attribué à la note r_{vi} .

Cette équation permet de représenter :

- les modèles utilisateur-utilisateur,
- les modèles item-item,
- les variantes normalisées ou pondérées (Z-score, amplification),
- les versions avec centrage ou non-centrage,
- ainsi que les modèles hybrides.

Cas des méthodes basées sur utilisateurs Dans les modèles use-based :

- les paires (v, i) correspondent à des utilisateurs v similaires ayant noté l'item j ;
- le voisinage N_{uj} prend la forme :

$$N_{uj} = \{(v, j) : v \in P_u(j)\};$$

- les poids sont :

$$b_{uj}(v, j) = \frac{\text{Sim}(u, v)}{\sum_{v' \in P_u(j)} |\text{Sim}(u, v')|}.$$

Le terme d'ajustement est typiquement :

$$a_{uj} = \mu_u.$$

Cas des méthodes item-based Pour les modèles item-based :

- les paires (v, i) correspondent aux items i similaires à j évalués par l'utilisateur u ;
- le voisinage est :

$$N_{uj} = \{(u, i) : i \in Q_j(u)\};$$

- les poids :

$$b_{uj}(u, i) = \frac{\text{Sim}(j, i)}{\sum_{i' \in Q_j(u)} |\text{Sim}(j, i')|}.$$

Selon les cas, a_{uj} peut être :

$$a_{uj} = 0, \quad \text{ou} \quad a_{uj} = \mu_u.$$

Avantage de cette vue unifiée. Cette formulation permet de :

- comprendre que les deux approches reposent sur le même principe fondamental ;
- comparer et analyser facilement les différences structurelles ;
- concevoir de nouvelles variantes hybrides ;
- interpréter les paramètres de manière systématique.

Ainsi, les méthodes basées sur les utilisateurs et celles basées sur les items ne sont que deux instanciations d'un modèle linéaire plus général appliqué à la matrice de notations.

Ainsi, bien que les méthodes de voisinage constituent une base intuitive et historiquement essentielle du filtrage collaboratif, leurs limites face à la sparsité et à la montée en dimension conduisent naturellement à considérer des approches fondées sur des modèles latents, présentées dans le chapitre suivant.

Chapitre 2

Filtrage collaboratif basé sur les modèles

2.1 Introduction

Bien que les approches basées sur la mémoire soient simples à mettre en oeuvre et efficaces lorsque la matrice des évaluations est dense, elles présentent plusieurs limites :

- une faible capacité de généralisation lorsque de nouveaux utilisateurs ou items apparaissent (problème du *cold start*) ;
- une forte dépendance à la densité des données ;
- une complexité croissante avec la taille du jeu de données.

Afin de pallier ces contraintes et d'améliorer la robustesse des prédictions, on introduit des approches **basées sur un modèle**. Ces dernières cherchent à construire une représentation latente des utilisateurs et des items, permettant d'apprendre les interactions sous-jacentes entre eux à partir des données existantes.

2.2 Factorisation de matrices non négatives

2.2.1 Formalsation et motivation

La factorisation des matrices non-négatives(NMF) a été introduite par Paatero et Tapper[10] dans le cadre de données environnementales, sous le nom de *positive matrix factorization*. L'idée est de relâcher la contrainte d'orthogonalité de la SVD(décomposition en valeurs singulière) et de la remplacer par la seule contrainte de non-négativité sur les facteurs latents. Mathématiquement, on part d'une matrice $R \in \mathbb{R}_+^{m \times n}$ très creuse que l'on essaie d'approximer par deux matrices $U \in \mathbb{R}_+^{m \times k}$ et $V \in \mathbb{R}_+^{k \times n}$ avec $k \ll \min(m, n)$, telles que :

$$X \approx UV^T$$

où U correspond à la matrice des utilisateurs avec les facteurs latents et V la matrice des items avec les facteurs latents. La note prédite de u pour i est alors :

$$\hat{R}_{m,n} = (UV^T)_{m,n} = \sum_{\ell=1}^k U_{m,\ell} V_{\ell,n},$$

une somme des k facteurs latents .

2.2.2 Problème d'optimisation

La principale différence avec les autres formes de factorisation de matrices est que les facteurs U et V doivent être non négatifs. Par conséquent, la formulation du problème d'optimisation pour la factorisation de matrices non négatives est donnée comme suit :

$$\min_{\substack{U, V \geq 0 \\ \text{rg}(U)=k, \text{rg}(V)=k}} L(R, UV) + \lambda_1 P(U) + \lambda_2 P(H) \quad (1)$$

où (λ_1, λ_2) sont des paramètres régularisation , k facteur latent et $P(\cdot)$ est une fonction de pénalité. Cette dernière mesure la précision de la prédiction de R par UV .

Fonctions de perte

Deux fonctions de perte peuvent être utilisées pour mesurer l'erreur de reconstruction, à savoir :

Moindres carrés (norme de Frobenius).

$$\mathcal{L}_{\text{LS}}(U, V) = \frac{1}{2} \sum_{(m,n) \in \Omega} (R_{m,n} - (UV)_{m,n})^2 = \frac{1}{2} \|R - UV^T\|^2 \quad (2)$$

où $\Omega \subseteq \{1, \dots, m\} \times \{1, \dots, n\}$ correspond à l'ensemble des entrées observées. Cette fonction découle d'une hypothèse de bruit gaussien additif sur les notes.

Divergence de Kullback-Leibler.

$$\mathcal{L}_{\text{KL}}(U, V) = \sum_{(m,n) \in \Omega} \left[R_{m,n} \log \frac{R_{m,n}}{(UV)_{m,n}} - R_{m,n} + (UV)_{m,n} \right]$$

Cette fonction de perte est la plus adaptée lorsque les données suivent un modèle de comptage (bruit de Poisson). Dans ce travail, nous nous limiterons à la perte quadratique (2) , qui constitue le choix le plus courant dans la littérature.

Régularisation

En ajoutant une régularisation de Ridge pour contrôler la complexité du modèle[2], l'équation (1) devient :

$$\min_{U \geq 0, V \geq 0} \frac{1}{2} \|R - UV^T\|^2 + \frac{\lambda_1}{2} \|U\|^2 + \frac{\lambda_2}{2} \|V\|^2, \quad (4)$$

où $\|\cdot\|^2$ désigne le carré de la norme de Frobenius.

2.2.3 Structure du problème

Le problème d'optimisation (4) est non-convexe par rapport aux variables (U, V) considérées conjointement. En effet, les termes $(UV)_{m,n}^2$ intervenant dans la fonction objectif sont des polynômes de degré 4 par rapport aux variables d'optimisation. Par conséquent, les méthodes d'optimisation classique ne permettent pas de garantir la convergence vers un minimum global. Néanmoins, ce problème possède une propriété de *biconvexité* : pour U fixé, la fonction $V \mapsto \mathcal{L}(U, V)$ est convexe et réciproquement, pour V fixé, la fonction $U \mapsto \mathcal{L}(U, V)$ est également convexe. Cette propriété est exploitée pour résoudre le problème d'optimisation à l'aide de différentes méthodes, notamment :

Règles multiplicatives de Lee & Seung

Dans leur article[7], Lee et Seung proposent une méthode d'optimisation adaptée au problème(4). La difficulté principale provient du fait de la positivité des matrices U et V qu'une descente de gradient ne permettrait pas de garder cette positivité. C'est pour cette raison que les auteurs introduisent une règle de mise à jour multiplicative qui préserve la positivité des matrices U et V . Pour comprendre l'origine de cette méthode, considérons la minimisation de la fonction objectif $\mathcal{L}$ par rapport à V , en supposant U fixé. Une descente de gradient classique consiste :

$$V \leftarrow V - \eta \nabla_V \mathcal{L}$$

où η est un paramètre d'apprentissage. Dans le cas du critère des moindres carrés régularisé, le gradient s'écrit :

$$\nabla_V \mathcal{L} = U^T (UV - X) + \lambda_2 V = \underbrace{U^T UV}_P + \lambda_2 V - \underbrace{U^T X}_Q.$$

Les matrices P et Q sont à coefficients positifs puisque : $U \geq 0$, $V \geq 0$ et $X \geq 0$. On peut donc écrire :

$$\nabla_V \mathcal{L} = P - Q.$$

La mise à jour de gradient devient alors :

$$V \leftarrow V - \eta(P - Q).$$

Cependant, pour un pas global η , certaines entrées de la matrice peuvent devenir négatives, ce qui viole la contrainte de non-négativité. L'idée introduite par Lee et Seung consiste à remplacer le pas global η par un pas adaptatif dépendant de chaque entrée de la matrice. Plus précisément, pour chaque coefficient $V_{\ell,i}$ de V , on choisit :

$$\eta_{\ell,i} = \frac{V_{\ell,i}}{P_{\ell,i}}.$$

En substituant ce choix dans la mise à jour du gradient, on obtient :

$$\begin{aligned} V_{\ell,i} &\leftarrow V_{\ell,i} - \frac{V_{\ell,i}}{P_{\ell,i}}(P_{\ell,i} - Q_{\ell,i}) \\ &= V_{\ell,i} \frac{Q_{\ell,i}}{P_{\ell,i}}. \end{aligned}$$

La mise à jour devient ainsi multiplicative plutôt qu'additive. Cette structure présente un avantage important : Si $V_{\ell,i}^{(t)} > 0$, alors

$$V_{\ell,i}^{(t+1)} = V_{\ell,i}^{(t)} \frac{Q_{\ell,i}}{P_{\ell,i}} > 0.$$

puisque le rapport $\frac{Q_{\ell,i}}{P_{\ell,i}}$ est positif. La non-négativité des matrices est donc automatiquement préservée au cours de l'optimisation, sans qu'il soit nécessaire d'effectuer une projection.

En développant les expressions de P et Q , on obtient les règles multiplicatives explicites utilisées dans la pratique :

$$V_{\ell,n} \leftarrow V_{\ell,n} \cdot \frac{(U^\top R)_{\ell,n}}{(U^\top U V + \lambda_2 V)_{\ell,n}}, \quad (2.1)$$

$$U_{m,\ell} \leftarrow U_{m,\ell} \cdot \frac{(R V^\top)_{m,\ell}}{(U V V^\top + \lambda_1 U)_{m,\ell}}. \quad (2.2)$$

Les divisions sont effectuées entrée par entrée. En pratique, une petite constante $\varepsilon > 0$ est ajoutée au dénominateur afin d'éviter toute division par zéro.

Lee et Seung montrent également que ces règles multiplicatives possèdent une propriété importante : la fonction objectif décroît de manière monotone au cours des itérations,

$$\mathcal{L}(U^{(t+1)}, V^{(t+1)}) \leq \mathcal{L}(U^{(t)}, V^{(t)}).$$

La démonstration repose sur l'introduction d'une fonction auxiliaire majeure la fonction objectif, ce qui conduit à un schéma d'optimisation de type *majorisation-minimisation* (MM)[8]

Cette méthode présente plusieurs avantages pratiques : elle est simple à implémenter, garantit la préservation de la non-négativité et assure une décroissance monotone de la fonction objectif. Néanmoins, elle converge généralement vers un point stationnaire des conditions KKT plutôt que vers un minimum global, et sa convergence peut être relativement lente.

De plus, toute entrée initialisée à zéro dans U et V reste nulle pendant toute l'optimisation, ce qui constitue l'une des principales limitations pratiques de cette approche.

Moindres carrés alternés non négatifs (ALS)

Les méthodes fondées sur la descente de gradient, qu'elles soient multiplicatives ou stochastiques, reposent toutes sur des mises à jour approximatives des variables d'optimisation. Elles exploitent le gradient de la fonction objectif mais ne tirent pas directement parti d'une propriété structurelle importante du problème de factorisation matricielle non négative : sa *biconvexité*. En effet, lorsque la matrice V est fixée, la fonction objectif $\mathcal{L}(U, V)$ est convexe par rapport à U . Réciproquement, lorsque U est fixé, $\mathcal{L}(U, V)$ est convexe par rapport à V . L'algorithme des moindres carrés alternés (ALS) exploite précisément cette propriété en résolvant successivement les deux sous-problèmes convexes obtenus en fixant alternativement l'une des matrices. Plus précisément, lorsque V est fixé, le problème (4) devient :

$$\min_{U \geq 0} \frac{1}{2} \|R - UV^T\|^2 + \frac{\lambda_1}{2} \|U\|^2.$$

En appliquant la décomposition de la norme de Frobenius sur les lignes, l'optimisation en U devient :

$$\min_{U \geq 0} \sum_{i=1}^m \left(\frac{1}{2} \|r_i - V u_i\|^2 + \frac{\lambda_1}{2} \|u_i\|^2 \right)$$

Chaque terme de la somme ne dépend que de u_i : i ème ligne de U . Minimiser cette somme revient donc à minimiser chaque terme séparément. On obtient m sous-problèmes indépendants :

$$\min_{u_i \geq 0} \frac{1}{2} \|r_i - V u_i\|^2 + \frac{\lambda_1}{2} \|u_i\|^2, \quad i = 1, \dots, m.$$

En faisant de même, lorsque U est fixé, on obtient :

$$\min_{v_j \geq 0} \frac{1}{2} \|r_j - v_j\|^2 + \frac{\lambda_2}{2} \|v_j\|^2, \quad j = 1, \dots, n.$$

Afin d'assurer la non-négativité, dans le cas d'éventuelle entrée négative, il suffit de

projeter sur le cône positif en remettant à zéro les entrées négatives :

$$u_i \leftarrow \max(u_i^*, 0), \quad v_j \leftarrow \max(v_j^*, 0)$$

où u_i^* et v_j^* correspondent respectivement aux solutions analytiques des deux sous-problèmes.

Propriétés théoriques, avantages et limites

À chaque étape de l'ALS, la fonction objectif est décroissante : $\mathcal{L}(U^{(t)}, V^{(t)}) \leq \mathcal{L}(U^{(t-1)}, V^{(t-1)})$, car chaque étape résout exactement un sous-problème convexe qui ne peut qu'abaisser $\mathcal{L}$. Tout point d'accumulation de la suite $(U^{(t)}, V^{(t)})$ est un point stationnaire des conditions KKT[1] ; la convergence vers un minimum global n'est cependant pas garantie en raison de la non-convexité conjointe du problème.

Sur le plan des avantages, l'ALS résout chaque sous-problème jusqu'à son optimum exact contrairement aux règles multiplicatives de Lee & Seung qui n'effectuent qu'un pas de gradient approché ce qui accélère la convergence en nombre d'itérations [4]. Il ne souffre pas non plus du problème des points absorbants : là où les règles multiplicatives maintiennent définitivement à zéro toute entrée nulle de U ou V , la résolution du système linéaire peut produire une valeur positive à cet endroit, élargissant ainsi l'espace des solutions explorées. Enfin, les m sous-problèmes en u_i étant indépendants à V fixé et de même pour les n sous-problèmes en v_i à U fixé l'algorithme se parallélise naturellement, propriété exploitée notamment dans Spark/MLlib [3].

Ses limites sont néanmoins réelles. La solution obtenue dépend de l'initialisation de $U^{(0)}$ et ne constitue qu'un optimum local ; il est donc recommandé de lancer plusieurs exécutions avec des initialisations aléatoires différentes et de retenir la meilleure. Par ailleurs, pour toute matrice diagonale inversible positive D de taille $k \times k$, les paires $(UD, D^{-1}V)$ produisent le même produit $UV^\top$, ce qui rend la factorisation non unique sans contrainte supplémentaire. Le choix du rang k reste également délicat : aucun critère universel n'existe, et les heuristiques disponibles (coefficient cophénétique, part de variance expliquée) cèdent souvent la place à l'interprétabilité des facteurs [4].

2.3 Complétion de matrices

La complétion de matrices constitue une approche fondamentale pour le problème de prédiction des valeurs manquantes dans les systèmes de recommandation. Elle repose sur l'hypothèse que la matrice utilisateur objet sous-jacente est de rang faible, et que les interactions observées suffisent à reconstruire l'ensemble de la matrice.

Soit $R \in \mathbb{R}^{u \times d}$ la matrice des interactions, et soit $\Omega \subseteq \{1, \dots, u\} \times \{1, \dots, d\}$ l'ensemble des indices correspondant aux entrées observées. Le problème de complétion de matrices consiste à retrouver une matrice X telle que :

$$X_{ij} = R_{ij}, \quad \forall (i, j) \in \Omega, \quad (2.3)$$

tout en imposant que X soit de rang faible. Ce problème peut être formulé comme :

$$\min_X \text{rang}(X) \quad \text{sous la contrainte} \quad X_{ij} = R_{ij}, \quad \forall (i, j) \in \Omega. \quad (2.4)$$

Cependant, la minimisation directe du rang est un problème NP-difficile. Une relaxation convexe classique consiste à remplacer la fonction rang par la norme nucléaire, définie comme la somme des valeurs singulières de la matrice :

$$\min_X \|X\|_* \quad \text{sous la contrainte} \quad X_{ij} = R_{ij}, \quad \forall (i, j) \in \Omega. \quad (2.5)$$

Cette formulation convexe a été largement étudiée et offre des garanties théoriques fortes. Sous certaines hypothèses de cohérence et lorsque le nombre d'observations est suffisant, la reconstruction exacte de la matrice de rang faible est possible avec une forte probabilité.

Dans le contexte des systèmes de recommandation, la complétion de matrices fournit un cadre théorique rigoureux pour analyser le problème de prédiction des préférences. Néanmoins, les méthodes d'optimisation associées, basées sur des algorithmes proximaux ou des décompositions en valeurs singulières répétées, sont généralement coûteuses en calcul et difficiles à déployer à grande échelle.

La factorisation matricielle peut être interprétée comme une reformulation paramétrique du problème de complétion de matrices. En représentant explicitement la matrice recherchée sous la forme $X = UV^T$, on sacrifie certaines garanties théoriques au profit d'une meilleure efficacité computationnelle. La NMF s'inscrit dans cette logique en ajoutant des contraintes de non-négativité, adaptées aux données de recommandation.

Chapitre 3

Système de recommandation basé sur l'apprentissage profond

Les chapitres précédents ont permis d'étudier les principales méthodes de filtrage collaboratif utilisées dans les systèmes de recommandation. Après avoir présenté les approches basées sur la mémoire, reposant sur les notions de similarité et de voisinage, nous avons introduit les méthodes basées sur des modèles, en particulier la factorisation de matrices et la complétion de matrices, qui offrent une meilleure capacité de généralisation face à la sparsité des données.

Dans la continuité de ces travaux, ce chapitre s'intéresse à l'utilisation de techniques d'apprentissage profond pour la construction de systèmes de recommandation. Ces approches visent à apprendre des représentations plus expressives des utilisateurs et des items, capables de modéliser des interactions complexes et non linéaires, dépassant les limites des modèles factorisés classiques.

Les méthodes d'apprentissage profond s'appuient sur des architectures de réseaux de neurones et sur des algorithmes d'optimisation efficaces, rendus praticables par les avancées récentes en matière de calcul, notamment grâce à l'utilisation des unités de traitement graphique (GPU). Ces évolutions ont fortement contribué à l'essor des systèmes de recommandation basés sur des modèles neuronaux.

Ce chapitre commence par un rappel des bases de l'apprentissage profond et des réseaux de neurones artificiels, avant de présenter une approche de filtrage collaboratif fondée sur ces principes, appelée filtrage collaboratif neuronal.

3.1 Bases de l'apprentissage profond (ANN)

L'apprentissage profond est un sous-type de l'apprentissage automatique, qui couvre essentiellement les algorithmes basés sur les réseaux de neurones artificiels (Artificial Neural Networks, ANN).

Un neurone artificiel réalise une transformation affine suivie d'une fonction d'activation :

$$h = \sigma(Wx + b)$$

où x est le vecteur d'entrée, W la matrice de poids, b le biais et σ la fonction d'activation.

La propagation avant (*forward propagation*) permet de calculer les prédictions du réseau, tandis que la rétropropagation (*backpropagation*) ajuste les paramètres afin de minimiser la fonction de perte via une descente de gradient :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}.$$

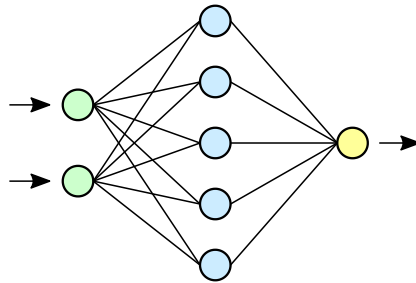


FIGURE 3.1 – Réseau de neurones

3.2 Filtrage collaboratif neuronal (NCF)

Les méthodes de filtrage collaboratif ont été les plus populaires pour la construction de systèmes de recommandation dans divers domaines. Des techniques populaires telles que la factorisation de matrices ont été largement utilisées, car elles sont faciles à implémenter

et fournissent des prédictions précises. Toutefois, ces derniers temps, grâce à de nouveaux domaines de recherche, les modèles basés sur l'apprentissage profond sont de plus en plus utilisés dans tous les domaines, y compris le filtrage collaboratif.

Le filtrage collaboratif neuronal (*Neural Collaborative Filtering*, NCF) est une méthode avancée de filtrage collaboratif renforcée par l'utilisation de réseaux de neurones. Dans la factorisation de matrices, la relation utilisateur-item est définie par le produit scalaire des matrices utilisateur et item. Dans le NCF, ce produit scalaire est remplacé par une structure de réseau de neurones. Par ce biais, le NCF cherche à atteindre deux objectifs :

- Généraliser la factorisation de matrices dans un cadre de réseau de neurones
- Apprendre les interactions ou relations utilisateur-item à travers un perceptron multicouche (*Multilayer Perceptron*, MLP)

La figure 3.2 montre la structure globale du NCF.

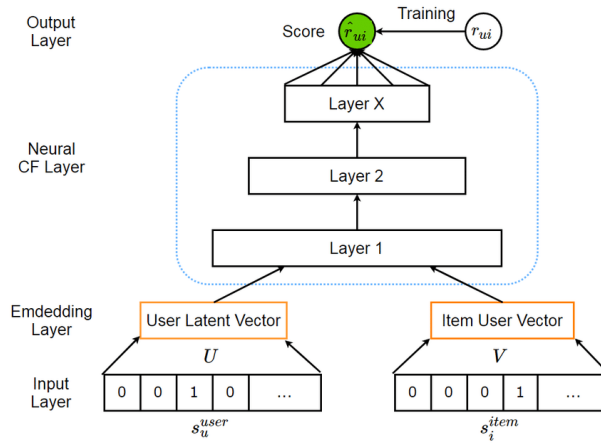


FIGURE 3.2 – Perceptron multicouche (MLP)

3.2.1 Formulation mathématique générale

On considère une matrice de notes :

$$R \in \mathbb{R}^{m \times n},$$

où m représente le nombre d'utilisateurs et n le nombre d'items. On note :

$$\Omega = \{(u, i) \mid r_{ui} \text{ est observé}\}$$

l'ensemble des interactions connues.

L'objectif est d'apprendre une fonction de prédiction :

$$\hat{r}_{ui} = f_{\theta}(u, i)$$

qui minimise l'erreur quadratique :

$$\min_{\theta} \sum_{(u,i) \in \Omega} (r_{ui} - \hat{r}_{ui})^2.$$

3.2.2 Embeddings utilisateurs et items

Dans NCF, chaque utilisateur et chaque item est représenté par un vecteur latent appris :

$$e_u \in \mathbb{R}^k, \quad e_i \in \mathbb{R}^k.$$

Ces vecteurs sont stockés dans deux matrices d'embeddings :

$$E_U \in \mathbb{R}^{m \times k}, \quad E_I \in \mathbb{R}^{n \times k}.$$

Contrairement à la factorisation matricielle classique où :

$$\hat{r}_{ui} = e_u^\top e_i,$$

le NCF remplace cette interaction linéaire par une fonction non linéaire :

$$\hat{r}_{ui} = f_{\theta}(e_u, e_i).$$

Ainsi, la factorisation matricielle apparaît comme un cas particulier du NCF.

3.2.3 Architecture du perceptron multicouche (MLP)

Les embeddings sont concaténés pour former le vecteur d'entrée :

$$x_{ui} = [e_u, e_i].$$

Le MLP applique ensuite une succession de transformations :

$$h^{(1)} = \sigma(W^{(1)}x_{ui} + b^{(1)}),$$

$$h^{(2)} = \sigma(W^{(2)}h^{(1)} + b^{(2)}),$$

$$\hat{r}_{ui} = W^{(L)}h^{(L-1)} + b^{(L)}.$$

Autrement dit, un perceptron multicouche (MLP) est un réseau de neurones composé de plusieurs couches entièrement connectées (c'est-à-dire que tous les nœuds de la couche

précédente sont connectés à tous les nœuds de la couche suivante). Chaque nœud d'un MLP utilise généralement une fonction sigmoïde comme fonction d'activation.

La fonction sigmoïde prend des valeurs réelles en entrée et renvoie une valeur réelle comprise entre 0 et 1 selon la formule suivante :

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}},$$

où x représente l'entrée.

Dans le NCF, la fonction d'activation utilisée est la fonction d'activation linéaire rectifiée (*Rectified Linear Unit*, ReLU). Cette fonction renvoie la même valeur si l'entrée est positive, et renvoie 0 si l'entrée est négative. La formule de la fonction ReLU est :

$$\text{ReLU}(x) = \max(0, x),$$

où x est l'entrée.

La figure 3.3 montre un perceptron multicouche (MLP).

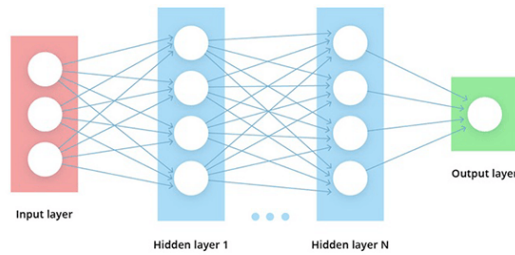


FIGURE 3.3 – Perceptron multicouche (MLP)

3.2.4 Fonction de perte et régularisation

Les paramètres du modèle sont :

$$\theta = \{E_U, E_I, W^{(l)}, b^{(l)}\}.$$

On minimise alors :

$$\min_{\theta} \sum_{(u,i) \in \Omega} (r_{ui} - f_{\theta}(e_u, e_i))^2 + \lambda (\|E_U\|_F^2 + \|E_I\|_F^2),$$

où λ est un coefficient de régularisation.

3.2.5 Interprétation théorique

Le NCF peut être interprété comme une généralisation non linéaire de la factorisation matricielle. Grâce aux fonctions d'activation et à la profondeur du réseau, il est capable d'apprendre des interactions complexes entre utilisateurs et items.

D'un point de vue géométrique, les embeddings projettent les utilisateurs et les items dans un espace latent où la proximité reflète la similarité comportementale. Le réseau neuronal applique ensuite une transformation non linéaire de cet espace afin d'améliorer la qualité des prédictions.

Après avoir présenté les bases de l'apprentissage profond, les principes des réseaux de neurones artificiels et le fonctionnement du filtrage collaboratif neuronal, la suite de ce chapitre s'attache à détailler les différents éléments constitutifs de cette approche et leur rôle dans la conception d'un système de recommandation fondé sur des modèles neuronaux.

Chapitre 4

Application des méthodes du filtrage collaboratif

4.1 Comparaison expérimentale des méthodes de recommandation

Afin d'évaluer les différentes approches étudiées, nous avons réalisé une expérimentation sur une matrice de notes de petite taille, composée de 5 utilisateurs et 6 items.

Une partie des notes observées est masquée aléatoirement selon un protocole de *hold-out*, puis utilisée comme jeu de test. Les modèles sont entraînés uniquement sur les notes restantes.

La qualité des prédictions est évaluée à l'aide de la métrique RMSE (*Root Mean Squared Error*) :

$$RMSE = \sqrt{\frac{1}{N} \sum_{(u,i) \in \Omega_{test}} (r_{ui} - \hat{r}_{ui})^2}$$

où :

- r_{ui} désigne la note réelle,
- $\hat{r}_{ui}$ la note prédite,
- Ω_{test} l'ensemble des notes masquées.

4.2 Résultats obtenus

Le tableau suivant présente les performances obtenues pour un taux de masquage de 25%.

Méthode	RMSE
User–User Collaborative Filtering	0.484
Item–Item Collaborative Filtering	2.496
NMF (Non-negative Matrix Factorization)	2.597
NCF (Neural Collaborative Filtering)	1.783

TABLE 4.1 – Comparaison des performances des différentes méthodes

4.3 Analyse des résultats

Les résultats montrent que la méthode **User–User** obtient la meilleure performance sur ce jeu de données avec un RMSE de 0.484.

Cette supériorité s’explique principalement par la petite taille de la matrice de notes, qui favorise les méthodes basées sur les voisinages lorsque plusieurs utilisateurs présentent des profils similaires.

Les méthodes **Item–Item** et **NMF** obtiennent des performances moins bonnes dans cette configuration, en raison du faible nombre d’items et du volume limité de données disponibles pour apprendre des structures latentes robustes.

La méthode **NCF**, bien qu’appuyée sur un modèle neuronal plus flexible, reste pénalisée par la faible quantité de données, les réseaux de neurones nécessitant généralement des jeux de données plus volumineux pour exploiter pleinement leur capacité de généralisation.

Ces observations confirment que le choix de la méthode dépend fortement de la taille et de la densité de la matrice utilisateur–item.

Conclusion générale

Ce projet annuel a permis d'étudier de manière progressive les principales approches de filtrage collaboratif utilisées dans les systèmes de recommandation, en mettant en évidence leurs fondements théoriques, leurs formulations mathématiques ainsi que leurs limites pratiques.

Dans un premier temps, les méthodes basées sur les voisinages ont été présentées à travers les approches utilisateur-utilisateur et item-item. Ces méthodes, historiquement parmi les plus anciennes, reposent sur l'idée que les préférences futures peuvent être estimées à partir des similarités observées entre utilisateurs ou entre items. Leur principal avantage réside dans leur simplicité d'interprétation et leur facilité de mise en œuvre, mais elles montrent rapidement leurs limites lorsque les données deviennent fortement creuses ou lorsque la taille des matrices augmente.

Dans un second temps, les approches basées sur les modèles ont été étudiées, en particulier la factorisation de matrices non négatives et la complétion de matrices. Ces méthodes permettent de projeter utilisateurs et items dans un espace latent de dimension réduite, ce qui améliore la capacité de généralisation et la robustesse face à la sparsité des données. La factorisation matricielle constitue aujourd'hui une base essentielle des systèmes de recommandation modernes en raison de son bon compromis entre précision et coût de calcul.

Enfin, le projet s'est ouvert aux méthodes récentes fondées sur l'apprentissage profond à travers le filtrage collaboratif neuronal (*Neural Collaborative Filtering*). Cette approche généralise la factorisation matricielle en remplaçant les interactions linéaires classiques par des architectures neuronales capables d'apprendre des relations non linéaires plus complexes entre utilisateurs et items. L'introduction des embeddings, des perceptrons multicouches et des fonctions de perte régularisées illustre l'évolution actuelle des systèmes de recommandation vers des modèles plus expressifs.

L'ensemble de ces travaux montre qu'aucune méthode ne constitue une solution universelle : le choix d'un modèle dépend du volume des données disponibles, du niveau de sparsité, des contraintes de calcul et du degré d'interprétabilité recherché.

En perspective, plusieurs prolongements peuvent être envisagés : l'évaluation expérimentale comparative des méthodes sur des bases de données réelles, l'intégration d'informations contextuelles, ou encore l'étude de modèles hybrides combinant filtrage collabo-

ratif, apprentissage profond et contenu explicatif.

Ainsi, ce projet met en évidence que les systèmes de recommandation constituent un domaine en constante évolution, au croisement de l'algèbre matricielle, de l'optimisation et de l'intelligence artificielle moderne.

Bibliographie

- [1] Charu C. Aggarwal. Recommender systems : The textbook, 2016.
- [2] Philippe Besse. Collaborative filtering, 2015.
- [3] Philippe Besse. Atelier sd2 : Recommandation par filtrage collaboratif (movielens), 2016.
- [4] Philippe Besse. Nmf : factorisation par matrices non négatives, 2016.
- [5] Emmanuel J. Candès and Terence Tao. The power of convex relaxation : Near-optimal matrix completion. *IEEE Transactions on Information Theory*, 56(5) :2053–2080, 2010.
- [6] Julien Delporte and Stéphane Canu. Apprentissage et factorisation matricielle pour la recommandation, 2012. Slides AAFD’2012, Université Paris 13.
- [7] Daniel D. Lee and H. Sebastian Seung. Learning the parts of objects by non-negative matrix factorization. *Nature*, 401 :788–791, 1999.
- [8] Jérémie Mary, Romaric Gaudel, and Philippe Preux. Bandits and recommender systems. In *First International Workshop on Machine Learning, Optimization, and Big Data (MOD’15)*, pages 325–336, Taormina, Italy, 2015.
- [9] Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. Spectral regularization algorithms for learning large incomplete matrices. *Journal of Machine Learning Research*, 11 :2287–2322, 2010.
- [10] Pentti Paatero and Unto Tapper. Positive matrix factorization : A non-negative factor model with optimal utilization of error estimates of data values. *Environmetrics*, 5(2) :111–126, 1994.